

Smart Hospital Patient & Resource Allocation System

NAME-E.A.P.B.Madanayaka

Index-AS20250517

GitHub Repository URL

Repository URL: <https://github.com/poornabuddhima/Smart-Hospital-Patient-Resource-Allocation-System>

Introduction

Private hospitals today offer inpatient treatment and doctor channeling services. Managing a hospital is a task. It involves different parts that need to work together smoothly. These include admitting patients using resources handling emergency cases and managing billing. Traditional hospital systems often struggle with problems.

- Patient registration is slow and full of mistakes. When staff have to handle everything by hand it leads to waits. Patients get frustrated. They may not be treated quickly. Their experience becomes poor.
- Finding a bed for a patient is another issue. Without a system that shows bed availability in time patients can be sent to rooms that are already full. Meanwhile other beds stay empty. This wastes space and delays care.
- In emergencies things get worse. The system doesn't always prioritize patients properly. A person in condition might not get help fast enough. That can mean life-or-death consequences.
- Billing is also unreliable. When charges are calculated manually mistakes happen. Fees for consultations room stays, emergency fees and discounts based on age are all subject to errors. This causes confusion and disputes.
- Hospital managers don't have data to make decisions. Without records it's hard to see how well the hospital is using its beds, what the revenue trends are or how many patients are coming in.
- Also data can be lost. If records are kept on paper they can be damaged, misplaced or destroyed. There's no backup. Information that should be kept safe can disappear at any time.
- All these problems show that hospitals need a way. A single software system that works for all parts of the hospital can help. It should be easy to use, with menus. It should manage admissions, assign beds handle billing and generate reports. When all these tasks are done in one place hospitals can run efficiently. Care becomes faster and safer.. Managers can make better decisions, with real data.

Objectives of the Project

The main goal of this project is to design and build a Smart Hospital & Resource Allocation System using the C programming language so that it can model the work of a hospital. The specific objectives are:

- To create a menu-driven program that uses core programming ideas such as functions, loops, if statements and 1D/2D arrays.
- To build a patient intake module that records details emergency triage rank, chosen specialty and ward admission information.
- To create a bed occupancy grid with a 2D array that shows which beds are free or full in time for all wards (General, Paediatric, Surgical and ICU).
- To automate billing so that base consultation fees, emergency surcharges, ward stay costs and age discounts are calculated automatically cutting out calculation mistakes.
- To create a priority patient system that identifies emergency triage levels (Normal, Urgent Critical) and allocates resources accordingly.
- To offer an analytics and reporting part that sums up patient numbers money earned, bed fill rates and records of the paying patients.
- To add file handling so that bed

- status and patient billing records survive between runs keeping data continuous.
- To promote software design habits, such, as modularity, reusable code, clear separation of parts and good documentation
- Doctor Channeling: Outpatient appointment booking with estimated consultation times and queue-based ticket generation.

System Architecture

1) Inter face

- Patient Admit
- Doctor channeling
- Available Beds
- Available doctors

2) Patient Admit

- New Patient Admit

Currently Admitted Patient

- Discharge And Patient Bill
- Return main manu

3) New Patient Admit

- Name
- Age
- Phone Number
- Check pateint
- Is Admitted to Ward? (1 = Yes, 0 = No)
- Get ID
- Ward
- Input patein previous admit diseases
- Emergency / Triage Level (1 = Normal, 2 = Urgent, 3 = Critical)
- Admit Date and Time
- Return main manu

4) Currently Admitted Patient

- Enter id
- How many days admit
- previous admit diseases
- Doctor checking time
- Input Medicine cost
- Input Surgry cost
- Adding some else

5) Discharge And Patient Bill

- Enter id
- Input Ward ID (1 to 4) and Days Admitted (integer).
- Set Days Admitted = 0 (Outpat
- Calculate full charge
- Get some discount

- Remove in ward
- Return main manu

6) Doctor channeling

- Select need consultant
- Check booking availability
- Enter name,age,phone number
- Get number 1-20 booking order
- Doctors check time start 4.pm
- Doctors average late time
- Your will come time

Scope of the Project — Detailed Explanation with Logic and Code

I am excited to share the Smart Hospital & Resource Allocation System, a console-based C application that simulates functional areas of hospital operations. In this guide I explain each area, its underlying logic. Provide the corresponding code implementation from the project.

Patient Registration

Logic

- Patient registration is the starting point of the system. When a new patient comes in the system must do the following:
- First it needs to find an empty spot in the parallel arrays. This is done by checking where ids[i] equals zero. Once an empty slot is found the system captures the patient's details. The name, age and phone number.
- Next the system determines if the patient will be admitted to a ward or treated as an outpatient (OPD). If admitted the system asks for the ward ID. Then searches for a free bed in that ward's 2D array. If a bed is available it marks the bed as occupied and saves the updated bed status to a file.
- The system also records the triage level. 1 For 2 for urgent 3 for critical. And updates the urgencyStats array for reporting and analysis.
- Then the system lets the user pick a specialty: 1 for OPD, 2 for Paediatrics 3 for Cardiology or 4 for Neurology.
- Finally the system records the admission date. Sets the initial values: days = 1 medical cost = 0, surgery cost = 0.

Code Implementation

```
void registerNewPatient(int ids[], int ages[], int wards[], int bedsAssigned[], int days[],
```

```
    char names[][30], char phones[][15], char history[][50],
```

```
    char admitDates[][25], int urgency[], int specialties[],
```

```
    float medCosts[], float surgeryCosts[], int beds[4][20],
```

```
    int *idCounter, int wardCapacities[], char* wardNames[],
```

```
    int *totalRegistered, int urgencyStats[]) {
```

```
    int foundIndex = -1;
```

```
    int run = 1;
```

```
    int isAdmitted = 0;
```

```
// Step 1: Find an empty slot in arrays
for (int i = 0; i < MAX_PATIENTS && run; i++) {
    if (ids[i] == 0) {
        foundIndex = i;
        run = 0;
    }
}

if (foundIndex != -1) {

    printf("\n--- New Patient Admit ---\n");

    // Step 2: Capture demographics
    printf("Enter Patient Name: ");
    scanf("%s", names[foundIndex]);

    printf("Enter Age: ");
    scanf("%d", &ages[foundIndex]);

    printf("Enter Phone Number: ");
    scanf("%s", phones[foundIndex]);

    // Step 3: Admission status
    printf("Is Admitted to Ward? (1 = Yes, 0 = No): ");
    scanf("%d", &isAdmitted);

    // Step 4: Assign unique ID
    ids[foundIndex] = *idCounter;
    printf("Patient ID : PAT-%d\n", ids[foundIndex]);
    (*idCounter)++;

    // Step 5: Bed allocation if admitted
    if (isAdmitted == 1) {
        printf("Enter Ward ID (1: General, 2: Paediatric, 3: Surgical, 4: ICU): ");
```

```

scanf("%d", &wards[foundIndex]);

if (wards[foundIndex] >= 1 && wards[foundIndex] <= 4) {
    int wardIdx = wards[foundIndex] - 1;
    int bedFound = 0;

    // Linear search for first available bed
    for (int b = 0; b < wardCapacities[wards[foundIndex]]; b++) {
        if (beds[wardIdx][b] == 0) {
            beds[wardIdx][b] = 1;          // Mark occupied
            bedsAssigned[foundIndex] = b + 1; // 1-based bed number
            bedFound = 1;
            printf("Bed Assigned: %d\n", bedsAssigned[foundIndex]);
            break;
        }
    }

    if (!bedFound) {
        printf("No beds available!\n");
        wards[foundIndex] = 0; // Fall back to OPD
    } else {
        saveBedStatus(beds, wardCapacities); // Persist to file
    }
} else {
    printf("Invalid Ward ID\n");
    wards[foundIndex] = 0;
}

} else {
    wards[foundIndex] = 0; // OPD — no ward
}

// Step 6: Medical history and triage
printf("Enter Patient Medical History: ");

```

```
scanf("%[^\\n]", history[foundIndex]);

printf("Enter Triage Level (1 = Normal, 2 = Urgent, 3 = Critical): ");
scanf("%d", &urgency[foundIndex]);

// Update analytics
(*totalRegistered)++;
if (urgency[foundIndex] >= 1 && urgency[foundIndex] <= 3) {
    urgencyStats[urgency[foundIndex]]++;
} else {
    urgencyStats[1]++;
}

// Step 7: Specialty selection
printf("Select Specialty ID (1: OPD, 2: Paediatrics, 3: Cardiology, 4: Neurology): ");
scanf("%d", &specialties[foundIndex]);

// Step 8: Admit date and initial values
printf("Enter Admit Date (e.g., 2026-06-01): ");
scanf("%s", admitDates[foundIndex]);

days[foundIndex] = 1;
medCosts[foundIndex] = 0;
surgeryCosts[foundIndex] = 0;

printf("Patient admitted successfully!\\n");
} else {
    printf("Hospital full! Maximum patient capacity reached.\\n");
}
}
```

```

=====
Yasaro Hospital Management System
=====
1. Patient Admit
2. Doctor Channeling
3. Available Beds
4. Available Doctors
5. Performance Reports & Analytics
Enter your choice: 1

--- Admit Patient Menu ---
1. New Admit Patient
2. Update Admitted Patient
3. Discharge & Print Patient Bill
4. Return to Main Menu
Choice: 1

--- New Patient Admit ---
Enter Patient Name: poo
Enter Age: 12
Enter Phone Number: 1111
Is Admitted to Ward? (1 = Yes, 0 = No): 1
Patient ID : PAT-1
Enter Ward ID (1: General, 2: Paediatric, 3: Surgical, 4: ICU): 1
Bed Assigned: 1
Enter Patient Medical History: 2
Enter Triage Level (1 = Normal, 2 = Urgent, 3 = Critical): 2
Select Specialty ID (1: OPD, 2: Paediatrics, 3: Cardiology, 4: Neurology): 2
Enter Admit Date (e.g., 2026-06-01): 2026-07-01
Patient admitted successfully!

```

2. Bed Allocation

Bed Allocation

Logic

Bed allocation is tightly coupled with registration but also operates independently for real-time display and discharge:

- 2D Matrix Representation: `int bedOccupancy[4][20]` where:
 - Row index = ward (0 = General, 1 = Paediatric, 2 = Surgical, 3 = ICU)
 - Column index = bed number
 - Value: 0 = Available, 1 = Occupied
- First-Fit Allocation: During registration, scan the row from index 0 upward; assign the first bed with value 0.
- Real-Time Display: `showAvailableBeds()` prints the matrix row by row, limited by each ward's capacity.
- Release on Discharge: `dischargePatient()` resets the bed value to 0 and re-saves the file.
- Capacity Enforcement: Loop bounds use `wardCapacities[wardId]` to respect each ward's actual size (General = 20, Paediatric = 10, Surgical = 10, ICU = 5).

Code Implementation

Display Function:

c

```
void showAvailableBeds(int beds[4][20], char* wardNames[], int wardCapacities[]) {  
    printf("\n--- Available Beds ---\n");  
    for (int i = 0; i < 4; i++) {  
        int wardId = i + 1;  
        printf("%d. %s (Capacity: %d):\n",  
            wardId, wardNames[wardId], wardCapacities[wardId]);  
  
        // Iterate only up to actual capacity (not full 20)  
        for (int j = 0; j < wardCapacities[wardId]; j++) {  
            printf("%d ", beds[i][j]);  
        }  
        printf("\n");  
    }  
}
```

```
=====
Yasaro Hospital Management System
=====
1. Patient Admit
2. Doctor Channeling
3. Available Beds
4. Available Doctors
5. Performance Reports & Analytics
Enter your choice: 3

--- Available Beds ---
1. General Ward (Capacity: 20):
  1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
2. Paediatric Ward (Capacity: 10):
  0 0 0 0 0 0 0 0 0 0
3. Surgical Ward (Capacity: 10):
  0 0 0 0 0 0 0 0 0 0
4. ICU (Intensive Care Unit) (Capacity: 5):
  0 0 0 0 0
```

3. Doctor Channeling

Logic

- Doctor channeling handles outpatient appointments without ward admission:
- Display specialties with fees, consultation times and daily caps.
- Check cap. If specialtyNumbers[specialtyId] >= specCaps[specialtyId] reject the booking.
- Increment queue counter for the selected specialty.

- Capture patient details (name, age, phone).
- Compute estimated arrival time using:
 - Start time = 16:00 (4:00 PM)
 - Late time = 15 minutes
 - Wait offset = $(\text{queueNumber} - 1) \times \text{consultationDuration}$
 - Total minutes = start + late + offset
- Convert to 12-hour AM/PM format for display.
- Print appointment ticket, with queue number, estimated time and fee.

Code Implementation

```
void doctorChanneling(char* specNames[], float specFees[], int specTimes[],
                    int specCaps[], int specialtyNumbers[]) {
    int specialtyId = 0;
    showAvailableDoctors(specNames, specFees, specTimes, specCaps);

    printf("Select Consultant Specialty ID (1-4): ");
    scanf("%d", &specialtyId);

    // Validate specialty ID
    if (specialtyId < 1 || specialtyId > 4) {
        printf("Invalid Specialty ID!\n");
        return;
    }

    // Check daily cap
    if (specialtyNumbers[specialtyId] >= specCaps[specialtyId]) {
        printf("Booking Full! Daily capacity for %s reached.\n",
            specNames[specialtyId]);
        return;
    }

    char name[30];
    int age = 0;
    char phone[15];

    printf("Enter Patient Name: ");
```

```

scanf("%s", name);

printf("Enter Age: ");

scanf("%d", &age);

printf("Enter Phone Number: ");

scanf("%s", phone);


// Increment queue counter

specialtyNumbers[specialtyId]++;

int queueNumber = specialtyNumbers[specialtyId];


// Time calculation

int startMinutes = 16 * 60;      // 4:00 PM

int avgLateTime = 15;           // 15 mins late start

int consultationDuration = specTimes[specialtyId];


int totalMinutes = startMinutes + avgLateTime
                    + ((queueNumber - 1) * consultationDuration);


int apptHour = (totalMinutes / 60) % 24;

int apptMin  = totalMinutes % 60;


// Convert to 12-hour format

int displayHour;

if (apptHour > 12) displayHour = apptHour - 12;

else if (apptHour == 0) displayHour = 12;

else displayHour = apptHour;


char* timePeriod = (apptHour >= 12) ? "PM" : "AM";


// Print ticket

printf("===== \n");

printf(" DOCTOR CHANNELING APPOINTMENT TICKET\n");

printf("-----\n");

```

```

printf("Specialty   : %s\n", specNames[specialtyId]);

printf("Patient Name : %s\n", name);

printf("Age & Phone   : %d Years | %s\n", age, phone);

printf("Booking Order: %02d\n", queueNumber);

printf("Consultation Start : 04:00 PM\n");

printf("Average Late Time : %d mins\n", avgLateTime);

printf("Estimated Arrival : %02d:%02d %s\n",

      displayHour, apptMin, timePeriod);

printf("Consultation Fee   : LKR %.2f\n", specFees[specialtyId]);

printf("===== \n");
}

```

```

--- Available Doctors & Specialties ---
1. General Practice (OPD) | Fee: LKR 1500.00 | Time: 15 mins | Daily Cap: 30
2. Paediatrics | Fee: LKR 2500.00 | Time: 20 mins | Daily Cap: 20
3. Cardiology | Fee: LKR 4500.00 | Time: 30 mins | Daily Cap: 12
4. Neurology | Fee: LKR 5000.00 | Time: 30 mins | Daily Cap: 10
Select Consultant Specialty ID (1-4): 2
Enter Patient Name: poo
Enter Age: 12
Enter Phone Number: 11
=====
DOCTOR CHANNELING APPOINTMENT TICKET
=====
Specialty      : Paediatrics
Patient Name   : poo
Age & Phone    : 12 Years | 11
Booking Order  : 01
Consultation Start : 04:00 PM
Average Late Time : 15 mins
Estimated Arrival : 04:15 PM
Consultation Fee : LKR 2500.00
=====

```

4. Billing & Invoicing

Logic

Billing is triggered at discharge and follows a strict formula-based pipeline:

Step	Formula
1. Base Consultation Fee	specFees[specialty]
2. Emergency Surcharge	Urgent → 20% of base; Critical → 50% of base

Step	Formula
3. Ward Stay Cost	$\text{days} \times \text{wardRates}[\text{ward}]$ (0 if not admitted)
4. Gross Total	$\text{Base} + \text{Surcharge} + \text{Ward Cost} + \text{Medicine} + \text{Surgery}$
5. Age Subsidy Discount	15% of gross if age < 5 OR age > 65; else 0
6. Final Payable	$\text{Gross} - \text{Discount}$
Each component is isolated in its own function for testability and reus	
Code Implementation	
<pre>// Step 1: Base consultation fee float baseConsultationFee(int specialty, float specFees[]) { if (specialty >= 1 && specialty <= 4) { return specFees[specialty]; } return 1500; // Fallback }</pre>	
<pre>// Step 2: Emergency surcharge float emergencySurcharge(int urgency, float baseFee) { float surcharge = 0; if (urgency == 2) { surcharge = baseFee * 0.20; // Urgent: +20% } else if (urgency == 3) { surcharge = baseFee * 0.50; // Critical: +50% } return surcharge; }</pre>	
<pre>// Step 3: Ward stay cost float wardDailyRate(int wardId, float wardRates[]) {</pre>	

```

    if (wardId >= 1 && wardId <= 4) {
        return wardRates[wardId];
    }
    return 0;
}

float wardCost(int isAdmitted, int wardId, int days, float wardRates[]) {
    if (isAdmitted > 0) {
        return days * wardDailyRate(wardId, wardRates);
    }
    return 0;
}

// Step 4: Gross total
float grossTotal(float baseFee, float surcharge, float wardCostVal,
    float medCostVal, float surgeryCostVal) {
    return baseFee + surcharge + wardCostVal + medCostVal + surgeryCostVal;
}

// Step 5: Age subsidy discount
float discount(int age, float grossTotalVal) {
    float discountAmount = 0;
    if (age < 5 || age > 65) {
        discountAmount = grossTotalVal * 0.15f;
    }
    return discountAmount;
}

```

```

-----
ID          : PAT-1
Name        : poor
Age         : 12 Years (Not Eligible)
Specialty   : General Practice (OPD)
Assigned Ward: General Ward (Bed 2)
Admit Date  : 2026-07-01
Discharge Date: 2026-07-16
Ward Stay   : 15 Days
Urgency Level: Level 1 (Routine)
-----

Base Consultation Fee : LKR 1500.00
Emergency Surcharge   : LKR 0.00 (0%)
Ward Stay Cost (15 Days) : LKR 45000.00
Medicine Cost         : LKR 0.00
Surgery Cost          : LKR 0.00
-----

Gross Total Bill      : LKR 46500.00
Age Subsidy Discount  : LKR          0.00
-----

Final Payable Amount  : LKR 46500.00
=====
Patient discharged successfully!

```

5. Performance Analytics

Logic

The analytics module provides a consolidated snapshot of hospital operations using the tracked global variables:

- **Urgency Breakdown** — counts per triage level (`urgencyStats[1..3]`).
- **Financial Summary** — cumulative revenue and discounts.
- **Bed Occupancy Percentage** — computed as $(\text{occupied} / \text{capacity}) \times 100$ for each ward, with a **guard against division by zero**.
- **Highest-Paying Patient** — name and bill amount tracked whenever a new bill exceeds the previous maximum.

```

void performanceReports(int totalRegistered, int urgencyStats[],
                        float totalRevenue, float totalDiscounts,
                        int beds[4][20], int wardCapacities[],
                        char* wardNames[], char highestName[],
                        float highestBill) {

    printf("===== \n");

```

```

printf(" PERFORMANCE REPORTS & ANALYTICS\n");
printf("===== \n");

// 1. Urgency breakdown
printf("1. Patient Urgency Breakdown:\n");
printf(" - Total Registered Patients : %d\n", totalRegistered);
printf(" - Level 1 (Routine)      : %d\n", urgencyStats[1]);
printf(" - Level 2 (Urgent)       : %d\n", urgencyStats[2]);
printf(" - Level 3 (Critical)      : %d\n", urgencyStats[3]);
printf("-----\n");

// 2. Financial summary
printf("2. Financial Summary:\n");
printf(" - Total Revenue Earned   : LKR %.2f\n", totalRevenue);
printf(" - Total Discounts Granted : LKR %.2f\n", totalDiscounts);
printf("-----\n");

// 3. Bed occupancy per ward
printf("3. Bed Occupancy Percentage Per Ward:\n");
for (int i = 0; i < 4; i++) {
    int wardId = i + 1;
    int capacity = wardCapacities[wardId];
    int occupiedCount = 0;

    for (int j = 0; j < capacity; j++) {
        if (beds[i][j] == 1) occupiedCount++;
    }

    // Guard against division by zero
    float occupancyPercentage = 0.0f;
    if (capacity > 0) {
        occupancyPercentage = ((float)occupiedCount / capacity) * 100.0f;
    }
}

```

```

printf(" - %s: %.1f%% (%d/%d Beds Occupied)\n",
       wardNames[wardId], occupancyPercentage,
       occupiedCount, capacity);
}
printf("-----\n");

// 4. Highest-paying patient
printf("4. Highest-Paying Patient Record:\n");
if (highestBill > 0.0f) {
    printf(" - Patient Name : %s\n", highestName);
    printf(" - Total Bill : LKR %.2f\n", highestBill);
} else {
    printf(" - No billed patients recorded yet.\n");
}
printf("=====\n");
}

```

```

=====
1. Patient Urgency Breakdown:
  - Total Registered Patients : 1
  - Level 1 (Routine)         : 1
  - Level 2 (Urgent)          : 0
  - Level 3 (Critical)        : 0
-----
2. Financial Summary:
  - Total Revenue Earned      : LKR 46500.00
  - Total Discounts Granted   : LKR 0.00
-----
3. Bed Occupancy Percentage Per Ward:
  - General Ward: 5.0% (1/20 Beds Occupied)
  - Paediatric Ward: 0.0% (0/10 Beds Occupied)
  - Surgical Ward: 0.0% (0/10 Beds Occupied)
  - ICU (Intensive Care Unit): 0.0% (0/5 Beds Occupied)
-----
4. Highest-Paying Patient Record:
  - Patient Name : poor
  - Total Bill   : LKR 46500.00
=====

```

6 File Persistence

Logic

Two files provide persistence across system runs:

1. `beds_status.txt` — stores the 2D bed matrix.
 - **Save:** Called after every bed assignment and discharge.
 - **Load:** Called once at program startup. If the file doesn't exist (first run), beds default to all-available.
2. `patient_records.txt` — append-only log of discharged patient bills.
 - **Append:** Called once per discharge, immediately after bill generation.

Code Implementation

Save Bed Status:

```
c
void saveBedStatus(int beds[4][20], int wardCapacities[]) {
    FILE *f = fopen("beds_status.txt", "w");
    if (f == NULL) {
        printf("Error: Could not open beds_status.txt for writing.\n");
        return;
    }
    for (int i = 0; i < 4; i++) {
        int wardId = i + 1;
        for (int j = 0; j < wardCapacities[wardId]; j++) {
            fprintf(f, "%d ", beds[i][j]);
        }
        fprintf(f, "\n");
    }
    fclose(f);
}
```

Load Bed Status (on startup):

```
c
void loadBedStatus(int beds[4][20], int wardCapacities[]) {
    FILE *f = fopen("beds_status.txt", "r");
    if (f == NULL) {
        return; // First run — no file yet
    }
}
```

```

    }
    for (int i = 0; i < 4; i++) {
        int wardId = i + 1;
        for (int j = 0; j < wardCapacities[wardId]; j++) {
            if (fscanf(f, "%d", &beds[i][j]) != 1) {
                break; // Corrupt / incomplete line
            }
        }
    }
    fclose(f);
}

```

Append Patient Billing Record:

```

c
void appendPatientRecord(int id, char* name, char* admitDate,
                        char* dischargeDate, float finalPayable) {
    FILE *f = fopen("patient_records.txt", "a"); // Append mode
    if (f == NULL) {
        printf("Error: Could not open patient_records.txt for appending.\n");
        return;
    }
    fprintf(f,
        "ID: PAT-%d | Name: %s | Admit Date: %s | "
        "Discharge Date: %s | Final Paid: LKR %.2f\n",
        id, name, admitDate, dischargeDate, finalPayable);
    fclose(f);
}

```